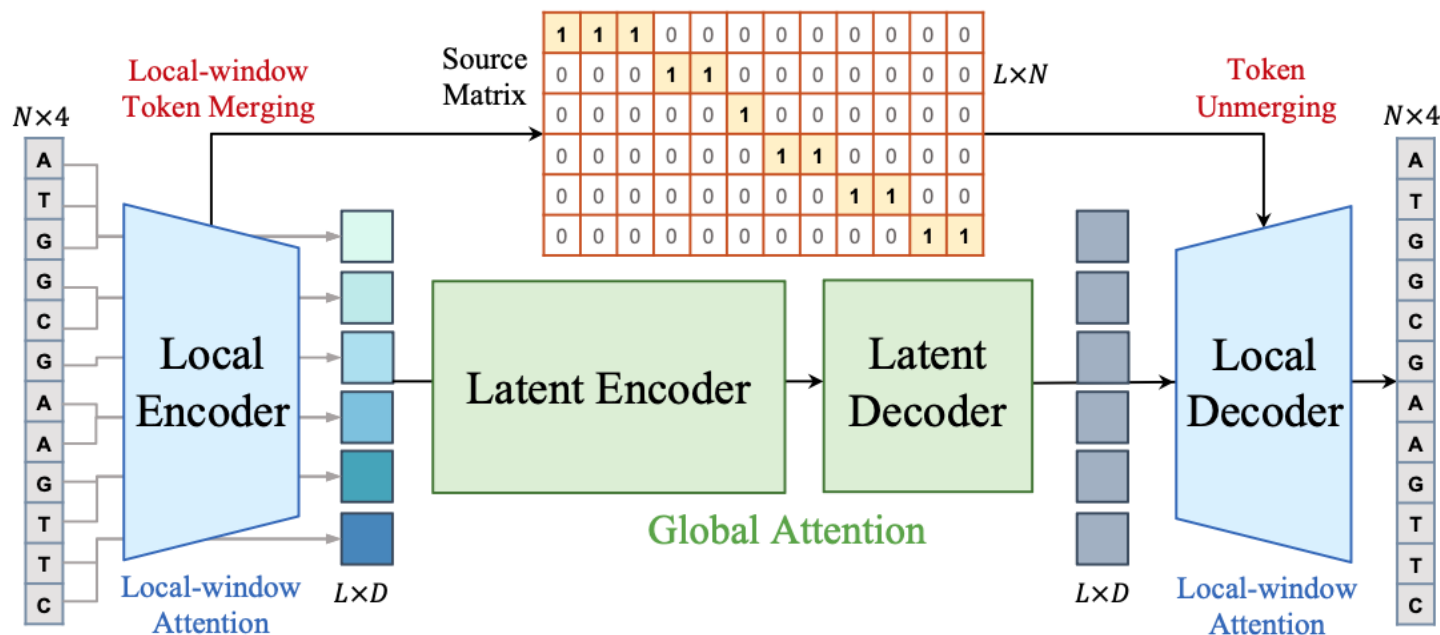


Summary

- MergeDNA uses a hierarchical AE transformer architecture.
- A local encoder-decoder surrounds a latent encoder-decoder structure. The local encoder-decoder implements adaptive DNA tokenisation (trains a differentiable DNA tokenizer).



Configuration	Local Enc.	Latent Enc.	Latent Dec.	Local Dec.
Embedding dim	1024			
Block number	4	20	4	2
Block type	Local-Attn	Attention	Attention	Local-Attn
Parameters	51M	253M	51M	25M
Optimizer	AdamW			
(β_1, β_2)	(0.9, 0.95)			
Training iterations	100,000			
Weight decay	1×10^{-4}			
Base learning rate	1×10^{-4}			
Batch size	256			
LR scheduler	Cosine Annealing			
Warmup iterations	10,000			
Gradient clipping	1.0			

Table A1: Configuration of the network architecture and pre-training for MergeDNA. The Local-Attn or Attention blocks denote the local-window or self-attention block with Flash-Attention implementation.

Blocks	Layers	Notes
Local Encoder	4	LocalToMeAttn blocks, window=16
Latent Encoder	20	Full attention (Flash Attn), LLaMA-style
Latent Decoder	4	Pre-training only
Local Decoder	2	Symmetric to Local Encoder
Total		~380 million params, D = 1024

Compression: local encoder $\rightarrow L = N/2$ tokens, latent encoder $\rightarrow K = L/2$ salient tokens.

Implementation

1. **Input:** nucleotides (ACTG) build coding and non-coding regions in a DNA sequence. This is converted into dense embeddings for the local encoder.
2. **Local encoder** for dynamic tokenisation for learning context segmentation (e.g. CDS vs nCDS) – local windows self attention layers of $N=16$.
 1. Token merging using ToMe matching based on the calculated token's cosine similarity score using the K vector from the attention mechanism.
 2. ToMe's bipartite soft matching algorithm connects the tokens from set A and B into r token edges, and the representations are merged using a soft merge function (weighted avg). The attention weights are proportional to the token size s:
$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} + \log s \right)$$
 3. Output from local encoder Z_L is half the size of the input N: $Z_L, S = \mathcal{E}_\phi(X)$.

Implementation

3. Latent Encoder: used for global context modelling and captures the long-range dependencies in the context input.

1. 20 transformer encoder blocks that learn the full attention of the merged tokens (Z_L). Embedding dimensions: 1024, number of heads: 16.
2. For longer sequences (e.g. nCDS) we use flash attention (is aware of input/output and speeds up training).
3. LLaMA (Touvron et al., 2023)
4. Output: Z'_L , produces context enriched token embeddings: $Z'_L = \mathcal{E}_\psi(Z_L)$,

4. Latent Decoder: token-level reconstruction

1. 4 transformer decoder blocks (+ same parameters as latent encoder). $\hat{Z}_L = \mathcal{E}_\omega(Z'_L)$
2. Only used during pre-training to assist the encoder and tokenizer.
3. Token unmerging is optionally implemented for K latent tokens $\rightarrow$ L positions.

Implementation

5. Local Decoder: for reconstruction from local decoder's output into original nucleotide sequence.

1. Token unmerging using the source matrix S to upsample decoded tokens from length L to original length N .
2. Output from this step (equation to implement): $\bar{Z}_N = S^T \hat{Z}_L$.
3. 2nd step – local attention stack ($n=2$) which mirrors the local encoder to get reconstructed nucleotide sequence $\hat{X}$: $\hat{X} = \mathcal{E}_\zeta(\bar{Z}_N)$.

6. Loss is important to model correctly:

1. Pass 1 uses Merged Token Reconstruction (MTR) Loss:
2. Pass 2 uses Latent MTR when the local encoder is frozen.
3. Adaptive Masked Token Modelling (AMTM) Loss

$$\mathcal{L}_{MTR}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log P(\hat{X}_i | X_i; \theta).$$

$$\mathcal{L}_{AMTM}(\theta) = -\frac{1}{K} \sum_{i: M_N(i)=1} \log P(\tilde{X}_i | X * M_N; \theta).$$

Combined: $\mathcal{L}_{\text{total}} = \mathcal{L}_{MTR}(\theta) + \lambda \mathcal{L}_{MTR}(\theta \setminus \{\phi\}) + \mathcal{L}_{AMTM}(\theta)$

(weight, lambda=0.25)

Loss used in each equation: cross-entropy